

BookStore (مكتبة البيان) — Audit & Upgrade Plan

Reference document. Generated audit of the current codebase, the dashboard-vs-back-office decision, the Cloudflare R2 integration plan, and a prioritized upgrade roadmap. Date: 2026-06-05 · Branch audited: `master` · Author: engineering audit

1. Executive summary

BookStore is a **single-store** Islamic/academic bookstore platform: a NestJS + Prisma/PostgreSQL backend and a React (Vite) RTL Arabic frontend. It already covers a full public storefront (catalog, academic taxonomy, book requests with WhatsApp redirect) and a complete admin back office (orders, books, catalog, academic, inventory, requests).

The codebase is **clean, well-organized, and feature-complete for an MVP**. The structure is modular and consistent. The gaps are not about features — they are about **production-readiness**: there is no authentication, no automated tests, no durable file storage, no containerization, and no observability.

The single most important finding: **the entire `/admin` surface is unauthenticated in code**. Anyone who can reach the server can read every order (with customer names, phones, and addresses) and mutate all data. This is a release blocker.

On the R2 question: R2 is **object storage**, not a database. It is the right home for **book-cover images** (which are currently written to the local filesystem and lost on every redeploy), but it is *not* a replacement for PostgreSQL. The database should move to a **managed Postgres** (Neon / Supabase / RDS). This document covers both.

2. Architecture snapshot (as built)

Backend — NestJS 11 + Prisma 5 + PostgreSQL

- Entry `src/main.ts`: global prefix `api`, global `ValidationPipe({ whitelist: true })`. Combined with per-controller `v1/...` prefixes this yields `/api/v1/...`.
- Modules under `src/modules/`:
- **Public**: `books/`, `academic/`, `requests/`, `import/` (CSV/XLSX), `images/` (cover auto-fill from Google Books / OpenLibrary), `image-sync/` (filename + Tesseract OCR matching), `clean/`.
- **Admin** (`src/modules/admin/`, mounted `/api/v1/admin/*`): `stats/`, `orders/`, `books/`, `catalog/`, `academic/`, `inventory/`, wired through `admin.module.ts` + `store-resolver.service.ts`.
- **Single-store model**: `StoreResolver` resolves one store from `STORE_SLUG` env or the first row, and caches it. The schema, however, is **multi-store-ready** (every domain row carries `storeId`).

Database — `prisma/schema.prisma`

- Core: `Store`, `Book`, `Inventory`, reference tables (`Author`, `Publisher`, `Country`, `Category`).
- Academic taxonomy: `Field` → `AcademicYear` → `Subject` → `BookOnSubject`.
- Leads/sales: `Request` (lead) → `Order` + `OrderItem`, with denormalized customer + money snapshots so historical orders survive later edits. Good modeling instinct.
- Enums for `InventoryStatus`, `RequestStatus`, `OrderStatus`. Sensible indexes on `storeId`-scoped lookups.
- 4 migrations present; schema is coherent.

Frontend — React 19 + Vite 8 + Tailwind 4 + TanStack Query

- Public pages: `home`, `search`, `book`, `academic/{fields,years,subjects,subject-books}`.
- Admin pages (`/admin`): `overview`, `orders`, `order-detail`, `books`, `catalog`, `academic`, `inventory`, `requests`.
- Shared libs: `queries.ts`, `admin-api.ts` (axios, `baseUrl: /api/v1/admin`), `types.ts`, `admin-types.ts`.
- Dev proxy forwards `/api` → `localhost:3000`.

Image / file handling (central to the R2 question)

- `images.service.ts`: fills missing `Book.imageUrl` by querying Google Books + OpenLibrary and storing **external URLs** (good — those are hotlinked, no storage cost, but also no control/permanence).
- `image-sync.service.ts`: scans a local `images/` folder, optionally OCRs covers (Tesseract `ara`), and **copies files into** `public/covers/`, serving them at `/covers/<file>` via `ServeStaticModule`.
- **Problem**: `public/covers/` is on the local disk and is not committed (and would not survive a container redeploy or a horizontally-scaled deployment). Locally-synced covers are therefore **ephemeral**. This is exactly what R2 solves.

3. Audit findings

Severity: ☐ blocker · ☐ high · ☐ medium · ☐ low / nice-to-have

3.1 Security & access control

- ☐ **Admin API is unauthenticated**. `/api/v1/admin/*` has no guard, no token, no session. CLAUDE.md acknowledges this ("gate behind a reverse proxy") but relying solely on a proxy is fragile — one misconfig exposes all customer PII (names, phones, wilaya, addresses on every `Order/Request`). **Add real auth before any public deployment**.
- ☐ **No rate limiting**. Public `requests` POST (lead creation) and `images/fill` (which fans out to external APIs) are abusable. Add `@nestjs/throttler`.
- ☐ **CORS is unconfigured**. Default NestJS = same-origin only; once frontend and API are on different origins in prod this needs an explicit allowlist (not `*`).
- ☐ **No security headers / helmet**. Add `helmet`.
- ☐ **Secrets hygiene**: `.env` is gitignored (good) and `.env.example` is minimal. Document every required var (see §6).

3.2 File / image storage → the R2 topic

- ☐ **Local-disk covers are not durable** (`public/covers/`). Lost on redeploy; breaks under >1 instance. **Move to R2** (§5).
- ☐ **Hotlinked external thumbnails** (Google/OpenLibrary) can rot or rate-limit. Optionally re-host the good ones in R2 for permanence.
- ☐ **No image optimization** (resize/webp). Covers are served at source resolution.

3.3 Database & data layer

- ☐ **`titleNormalized` is nullable and maintained ad-hoc**. Matching code falls back to `normalizeArabic(title)` at runtime. Consider always populating it on write so indexed search is reliable.
- ☐ **Arabic search uses `contains`** (sequential `ILIKE`), fine at current scale but won't scale. Postgres `pg_trgm` + GIN index, or full-text search, is the upgrade path.
- ☐ **No connection pooling strategy documented**. Serverless Postgres (Neon) needs a pooled connection string; document it.
- ☐ **Decimal money columns** are correctly typed `@db.Decimal(10,2)`. Good.

3.4 Backend code quality

- ☐ **Zero automated tests.** No `*.spec.ts` anywhere; Jest is configured but unused. The order/request conversion flow and the OCR matching pipeline are exactly the kind of logic that needs tests.
- ☐ **any types leak through `admin-api.ts`** (`createBook(data: any)`, `createOrder(data: any)`). Tighten to shared DTO types.
- ☐ **No global exception filter / structured error responses.** Errors surface as raw Nest defaults.
- ☐ Module boundaries, naming, and comments are consistently good. The image-sync strategy ladder is well-documented.

3.5 Frontend

- ☐ **No auth gate on `/admin` routes** (matches backend gap). Once backend auth exists, add a login screen + route guard.
- ☐ **No error boundaries / loading skeleton conventions** verified across admin pages.
- ☐ Modern stack (React 19, TanStack Query, Tailwind 4), clean component/lib split.

3.6 DevOps / observability

- ☐ **No CI.** No GitHub Actions, no lint/test/build gate on push.
- ☐ **No containerization.** No Dockerfile / compose; deployment story is undocumented.
- ☐ **No logging/monitoring** beyond Nest's default `Logger`. No error tracking (Sentry), no health-check endpoint, no metrics.
- ☐ **No `/health` readiness endpoint** for orchestration.

4. Strategic question: optimize the existing dashboard, or build a "back office"?

Short answer: you already have a back office — keep optimizing it; do not rebuild.

The current admin app is not a thin "dashboard" (read-only KPIs). It is already a transactional back office: orders with a status workflow, full book CRUD, catalog and academic taxonomy management, inventory, and lead-to-order conversion. Rebuilding from scratch would throw away working, well-structured code.

What "make it a proper back office" actually means here is **hardening + a few capability gaps**, not a rewrite:

Need	Status	Action
Authentication & roles (admin/staff)	☐ missing	P0 — add (see §6)
Audit log (who changed what)	☐ missing	P1 — track mutations on orders/inventory
Durable media storage	☐ ☐ local disk	P0/P1 — R2 (see §5)
Order workflow	☐ present	optimize: bulk actions, filters, CSV export
Inventory	☐ present	add low-stock alerts / reorder thresholds
Stats/analytics	☐ basic	richer charts, date-range, top-books
Multi-store	schema-ready, UI single-store	defer until needed

Recommendation: treat it as the back office it already is. Prioritize (1) auth, (2) durable storage, (3) tests/CI. Then iterate on operator-efficiency features (bulk actions, exports, audit log). A from-scratch rebuild is not justified.

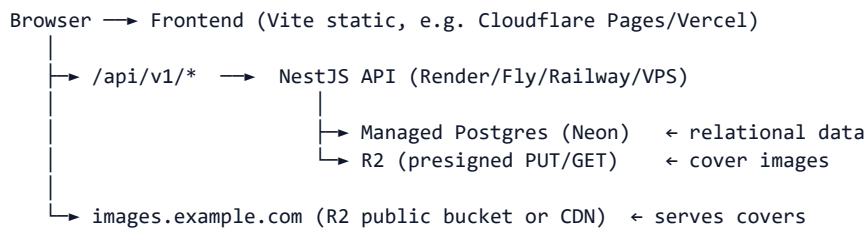
5. Cloudflare R2 integration plan

5.1 Clear up the terminology first

R2 is S3-compatible object storage — it stores files (images, PDFs), not relational data. It cannot replace PostgreSQL. So the request "integrate R2 for the database" splits into two independent decisions:

- 1 **Media files (book covers) → Cloudflare R2.** ☐ Correct fit. This is the real win.
- 2 **The relational database itself → managed Postgres** (Neon, Supabase, or AWS RDS). R2 is **not** an option here.
- 3 If you want the Cloudflare ecosystem for data too, the relational equivalent is **Cloudflare D1** (SQLite) — but migrating off Postgres/Prisma to D1 is a significant rewrite and loses Postgres features you already use (Decimal, enums, rich indexing). **Not recommended.** Keep Postgres; just host it on a managed provider.

5.2 Target architecture



5.3 R2 implementation steps (for the covers)

R2 is S3-compatible, so use the AWS SDK v3.

- 1 **Create the bucket** + an R2 API token (Access Key ID / Secret) in the Cloudflare dashboard. Optionally attach a custom domain (e.g. `covers.albayan.example`) or enable the public `r2.dev` URL.
- 2 **Add env vars** (see §6): `R2_ACCOUNT_ID`, `R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`, `R2_BUCKET`, `R2_PUBLIC_BASE_URL`.
- 3 **Add a `StorageModule`** wrapping `@aws-sdk/client-s3` configured for R2:

```
// endpoint: https://<ACCOUNT_ID>.r2.cloudflarestorage.com
// region: "auto", forcePathStyle: true
```

Expose `upload(key, buffer, contentType)` → returns `${R2_PUBLIC_BASE_URL}/${key}` and `getPresignedPutUrl(key)`.

- 1 **Rewrite `image-sync.service.ts` Step C:** instead of `fs.copyFileSync` into `public/covers/`, `upload()` the buffer to R2 and store the returned public URL in `Book.imageUrl`. Drop the `ServeStaticModule` covers dependency.
- 2 **(Optional) Re-host external thumbnails:** in `images.service.ts`, after fetching a Google/OpenLibrary URL, optionally download + push to R2 for permanence instead of hotlinking.
- 3 **(Optional) Admin direct upload:** add an endpoint that returns a **presigned PUT URL** so the admin UI can upload a cover straight to R2 (no large multipart through the API).
- 4 **One-time migration script:** walk existing `public/covers/*` and any hotlinked URLs, upload to R2, and rewrite `Book.imageUrl`.
- 5 **Image optimization (optional):** use Cloudflare Images or an on-the-fly transform so covers are served as resized WebP.

Tooling note: this workspace has a **Cloudflare MCP connector** (R2 bucket create/list/get + docs search) and a **Neon MCP connector** (project/branch/SQL). These can provision the bucket and the database branch directly when you're ready to execute.

5.4 Database hosting (the other half of "the database")

- Move `DATABASE_URL` to **Neon** (or Supabase). Use Neon's **pooled** connection string for the app and the **direct** one for `prisma migrate`.
- No schema changes required — Prisma + Postgres carries over 1:1.
- Set up a **staging branch** (Neon branching is ideal) so migrations are tested before `master`.

6. Prioritized upgrade roadmap

□ P0 — before any public/production deployment

- 1 **Admin authentication.** Add an auth layer (JWT or session) + a NestJS `AuthGuard` on every `/admin/*` controller, plus a login screen and route guard on the frontend. Add a `User/AdminUser` table (or use Supabase/Neon Auth). Roles: at least `admin`.
- 2 **Durable cover storage on R2** (§5.3) — stop writing to local disk.
- 3 **Managed Postgres** (§5.4) — move off any local/unmanaged DB; pooled connection string.
- 4 **Secrets + env documentation** — expand `.env.example`:

```
DATABASE_URL=          # Neon pooled
DIRECT_URL=            # Neon direct (migrations)
PORT=3000
STORE_SLUG=
R2_ACCOUNT_ID=
R2_ACCESS_KEY_ID=
R2_SECRET_ACCESS_KEY=
R2_BUCKET=
R2_PUBLIC_BASE_URL=
JWT_SECRET=
CORS_ORIGINS=
```

□ P1 — production hardening (first weeks)

- 1 **Rate limiting** (`@nestjs/throttler`) on public `requests` + `images/fill`.
- 2 **CORS allowlist** + **helmet** security headers.
- 3 **CI pipeline** (GitHub Actions): `install` → `lint` → `tsc` → `test` → `build`, on every PR to `develop/master`.
- 4 **First tests**: order/request conversion, money totals, the image-sync matching ladder, request creation validation.
- 5 **Dockerfile + compose** (api + postgres) for reproducible deploys.
- 6 `/health` endpoint + **error tracking** (Sentry) + a **global exception filter**.

□ P2 — quality & scale

- 1 **Audit log** for admin mutations (orders, inventory).
- 2 **Arabic search upgrade** — `pg_trgm` GIN index or Postgres FTS; always populate `titleNormalized` on write.
- 3 **Tighten any types** in `admin-api.ts` to shared DTOs.
- 4 **Back-office UX**: bulk order actions, CSV export, low-stock alerts, richer date-range analytics.
- 5 **Image optimization** (Cloudflare Images / WebP).

□ P3 — later

- 1 Multi-store activation (schema already supports it).
- 2 Customer accounts / order history (currently lead-only).
- 3 i18n beyond Arabic if ever needed.

7. Dependency / version notes

- Stack is current: NestJS 11, Prisma 5.22, React 19, Vite 8, Tailwind 4, TanStack Query 5 — no urgent upgrades.
- Prisma 6 exists; 5.22 is fine, upgrade opportunistically.

- `tesseract.js` (OCR) is heavy and runs in-process — fine for batch admin sync, but keep it off the hot request path (it already is).
-

8. One-paragraph answer to "what do we need to upgrade?"

You don't need to rebuild anything — the app is a well-structured, feature-complete MVP with a real back office already in place. What it needs to become production-grade, in order: **(1) authentication on the admin surface** (today it's wide open), **(2) durable image storage on Cloudflare R2** instead of the local `public/covers/` folder, **(3) a managed Postgres host** (Neon/Supabase) for the database — R2 does **not** replace the DB, it only stores files — and then **(4) the production basics**: rate limiting, CORS/helmet, CI, a test suite, Docker, and error monitoring. Do P0 before you ship; P1 in the first weeks after.